

1. Objetivos da Arquitetura

- Separar responsabilidades: apresentacao, negocio, persistencia.
- Permitir substituicao de SQLite por PostgreSQL sem alterar servicos.
- Garantir testabilidade dos servicos de forma isolada.
- Suportar multiplos usuarios simultaneos sem conflitos de sessao.

2. Suposicoes e Dependencias

- Python 3.11+ disponivel no ambiente de execucao.
- Streamlit gerencia ciclo de vida da interface e sessoes.
- SQLite adequado para demonstracao; PostgreSQL para producao.
- Usuarios acessam via navegador web moderno.

3. Arquitetura em Camadas

O KanBite adota arquitetura em 4 camadas:

Camada	Modulos	Responsabilidade
Apresentacao	pages/, components/, assets/	UI Streamlit, sessao, navegacao
Servicos	services/	Regras de negocio: WIP limit, timestamps, metricas
Repositorios	database/repos/	CRUD via SQLAlchemy ORM; sem logica de negocio
Dominio	models/	Entidades ORM: User, Board, Swimlane, Card
Banco de Dados	kanbite.db (SQLite)	Persistencia de dados

4. Componentes e Responsabilidades

Componente	Responsabilidade
app.py	Entry point; configura page_config e injeta CSS global
pages/01_login.py	Formulario de login; chama auth_service.autenticar()
pages/02_register.py	Formulario de cadastro; chama auth_service.registrar()
pages/03_metrics.py	Dashboard de metricas com Plotly
pages/04_board.py	Quadro Kanban: CRUD de cartoes, swimlanes, movimentacao
components/sidebar.py	Sidebar de navegacao com estado de autenticacao
components/kan_card.py	Componente visual de cartao (prioridade, prazo)
services/auth_service.py	Registrar e autenticar usuarios com bcrypt

services/kanban_service.py	CRUD de board/card/swimlane; valida WIP; transacoes
services/metrics_service.py	Calcula cycle time, lead time, throughput, WIP
database/engine.py	Engine SQLAlchemy e fabrica de sessoes
database/repos/	BoardRepo, CardRepo, SwimlaneRepo, UserRepo (so CRUD)
models/	Entidades ORM com relacionamentos e enums

5. Deciso es e Justificativas

Streamlit

Permite criar interfaces web interativas so com Python. Restricao: reexecuta o script a cada interacao; estado gerenciado via session_state.

SQLite

Zero configuracao, portavel como arquivo. Adequado para prototipo academico. Restricao: sem controle de concorrencia para multiplos escritores.

SQLAlchemy ORM

Abstrai dialeto SQL; troca de banco via connection string. Padrao da industria.

bcrypt

Hashing adaptativo resistente a ataques de forca bruta.

Padrao Repository

Isola acesso a dados; permite testes de servicos com mocks.

6. Mecanismos de Arquitetura

Mecanismo	Implementacao
Persistencia	SQLAlchemy + SQLite; sessao por get_session()
Autenticacao	bcrypt para hash; st.session_state para user_id
Validacao WIP	WipLimitError no service antes de qualquer chamada ao repo
Timestamps	CardRepo.mover_coluna() registra started_at e finished_at
Cascade delete	Definido nos relacionamentos ORM (cascade='all, delete-orphan')
Metricas	Calculadas em tempo real pela metrics_service sem cache

7. Impacto das Ferramentas na Arquitetura

Streamlit reexecuta o script Python a cada interacao. Estado deve ser gerenciado via st.session_state. A sessao do banco e recriada por get_session() a cada reexecucao. pytest exigiu que servicos recebam sessao como parametro (injecao de dependencia), resultando em codigo mais testavel e desacoplado.